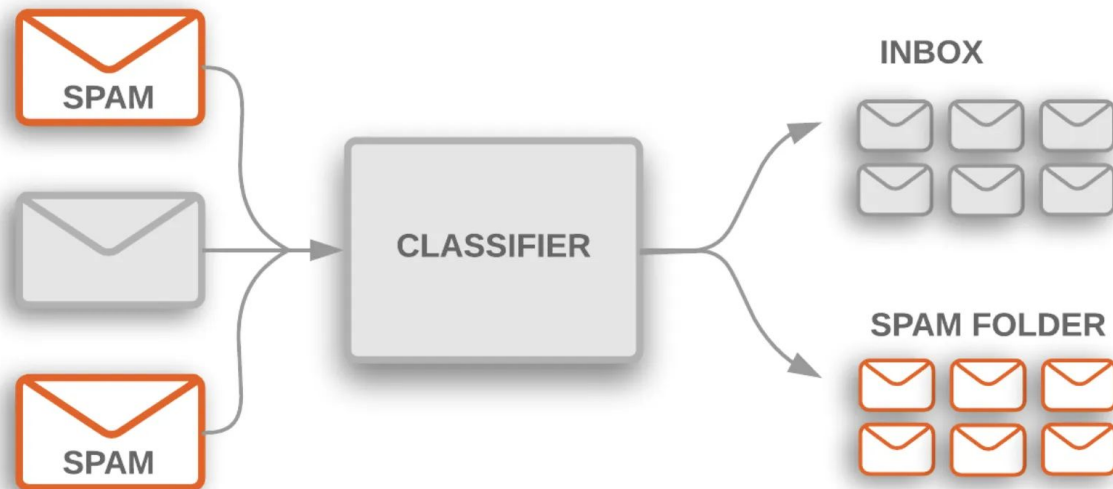


Machine learning

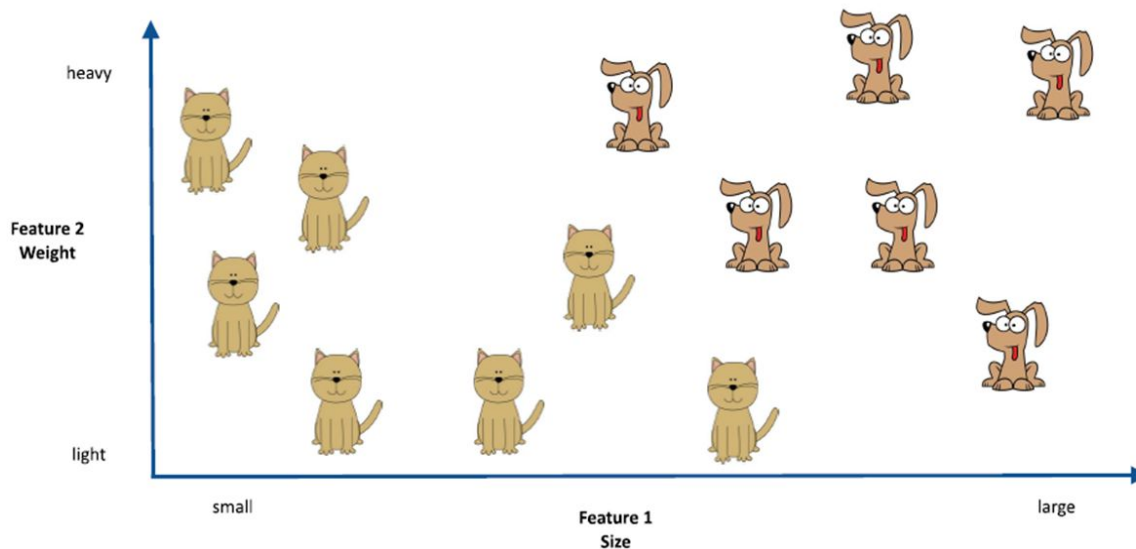
Lecture 5 - Classification

21/04/2025
Toker Gilat



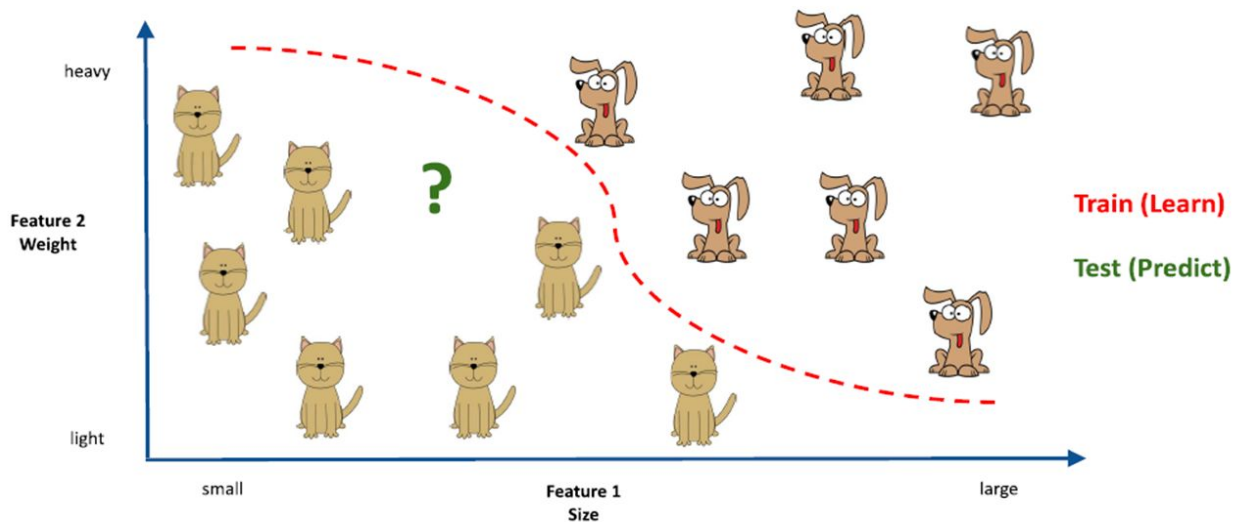
What is Classification

Classification = Learn to map from **observations** (features) to **categories** (classes)



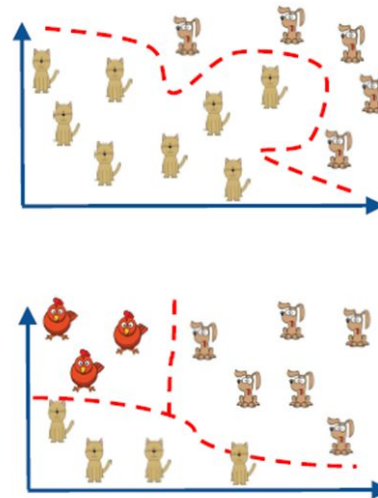
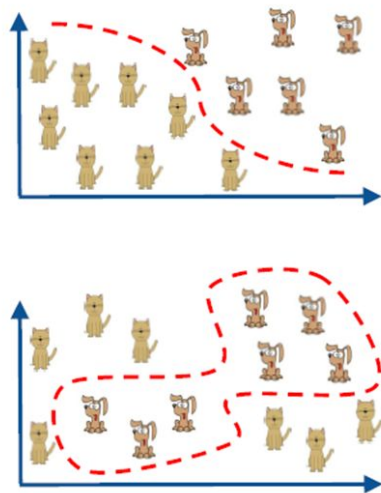
What is Classification

Classification = Learn to map from **observations** (features) to **categories** (classes)



What is Classification

Classification = Learn to map from **observations** (features) to **categories** (classes)



Supervised Learning I

27

Classification “types”

Binary
Classification



- Spam
- Not spam

Multiclass
Classification



- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Multi-label
Classification



- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Supervised learning

Learning from examples:

X : predictors/features

y : target(s)/label(s)

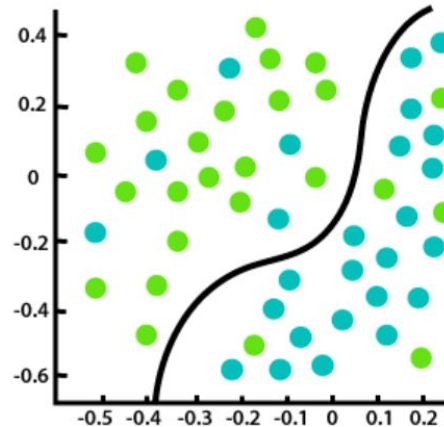
Goal: finding a function $f(X) \rightarrow y$

Train: $\text{train}(X, y) \rightarrow \text{"model"}$

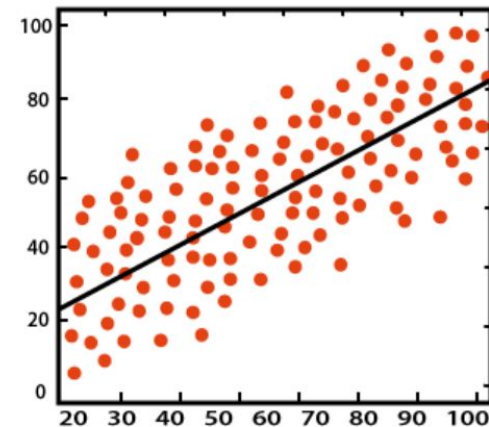
Predict: y_{red}

Evaluate: $\text{eval}(y_{\text{rue}}, y_{\text{red}}) \rightarrow \text{metrics}$

Types: Regression, Classification, and more...



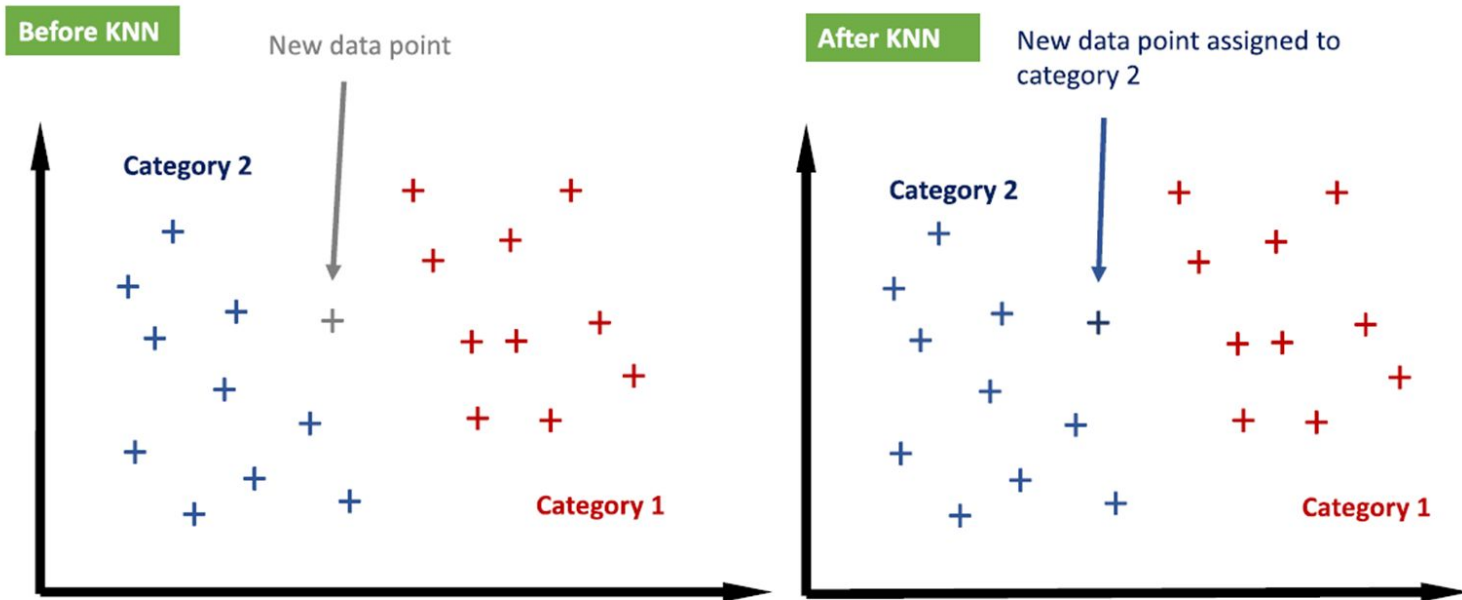
Classification



Regression

K - Nearest Neighbors (K-NN)

K - Nearest Neighbors (K-NN)



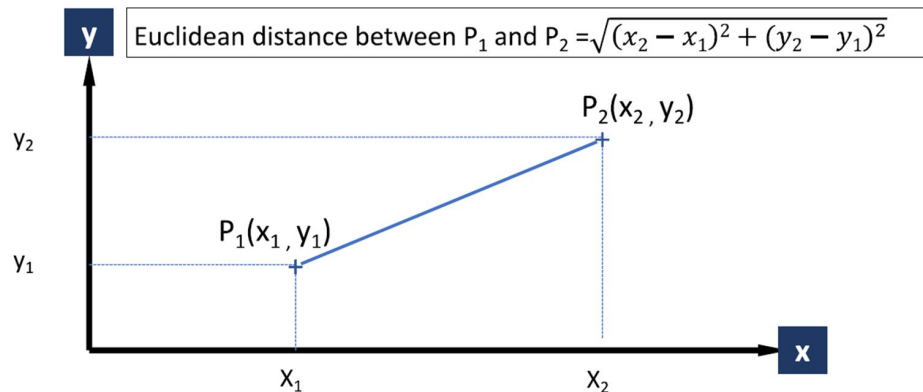
K - Nearest Neighbors (K-NN)

1. **Choose the number of neighbors (K):** Selecting the right value of K is crucial.
2. **Calculate Distance to Neighbors:** For each new data point, calculate the Euclidean distance (or another distance metric) to all points in the dataset.
3. **Count the Neighbors' Categories:** Among the K nearest neighbors, count how many belong to each category (or class).
4. **Assign the Category by Majority Vote:** Assign the new data point to the category that has the highest count (majority vote) among its K nearest neighbors

K - Nearest Neighbors (K-NN)

Calculate Distance to Neighbors

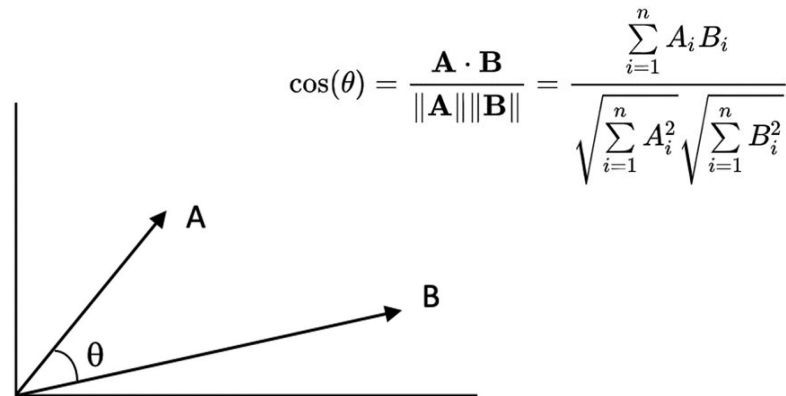
Euclidean distance



Supervised Learning I

52

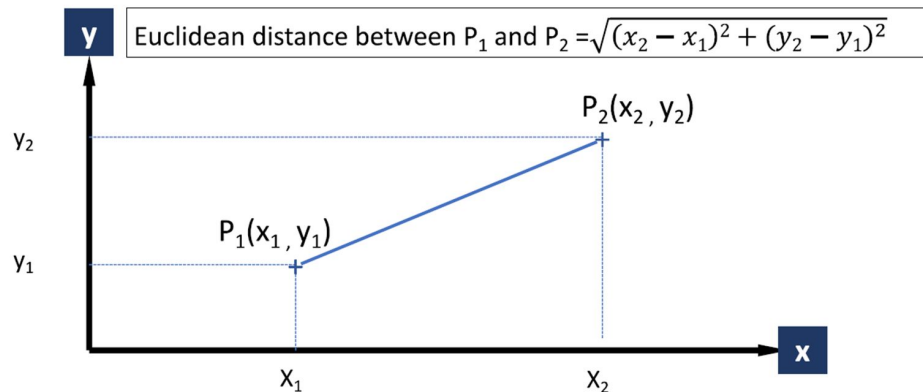
Cosine Similarity



K - Nearest Neighbors (K-NN)

Calculate Distance to Neighbors

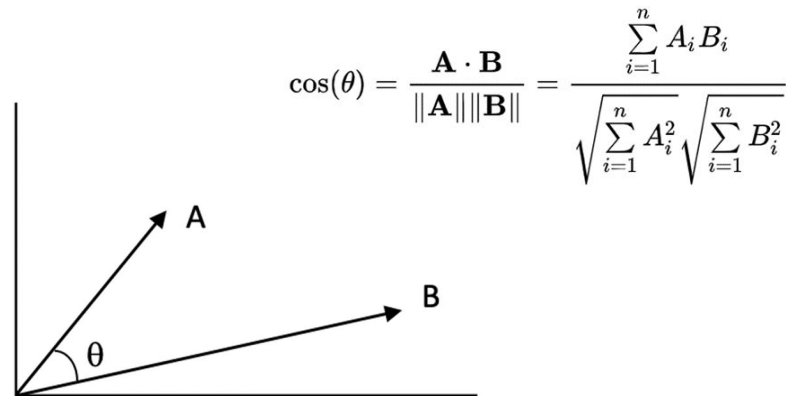
Euclidean distance



Supervised Learning I

52

Cosine Similarity



K - Nearest Neighbors (K-NN)

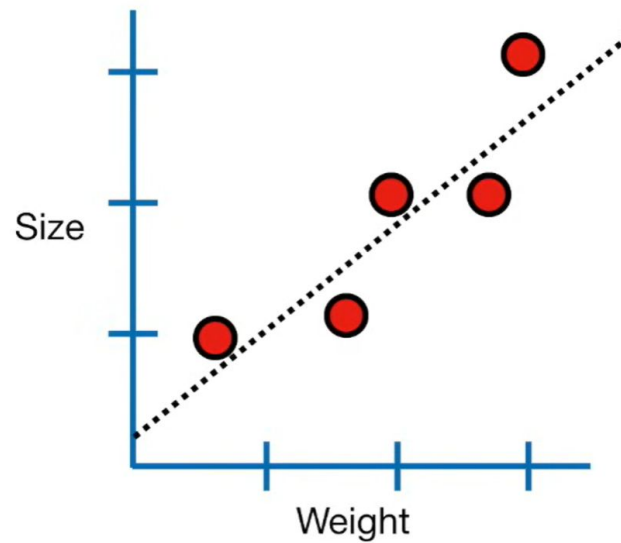
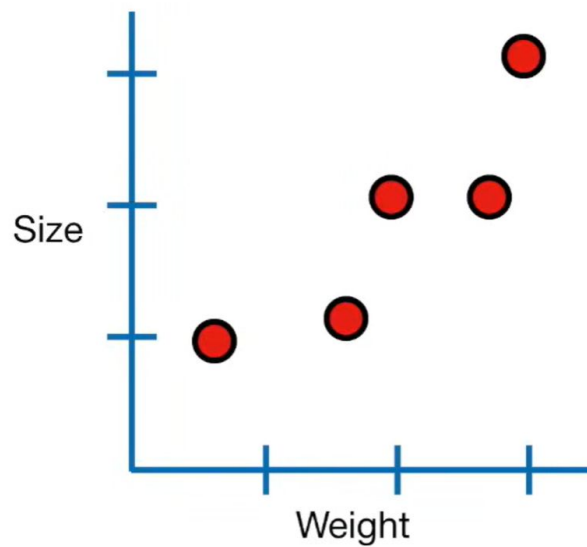
Calculate Distance to Neighbors

Metric	Euclidean Distance	Cosine Similarity
Focus	Absolute magnitude	Direction (angle)
Range	$[0, \infty)$ (always positive)	$[-1, 1]$
Scaling Required	Yes	No
Use Case	Physical distance, clustering	Text similarity, recommendations
Sensitive to Zero Vectors	Yes	Cannot handle zero vectors

Logistic regression

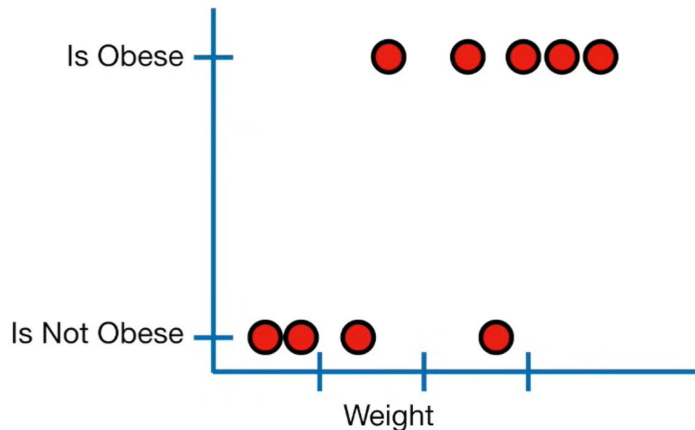
Linear Regression

Reminder: Linear Regression



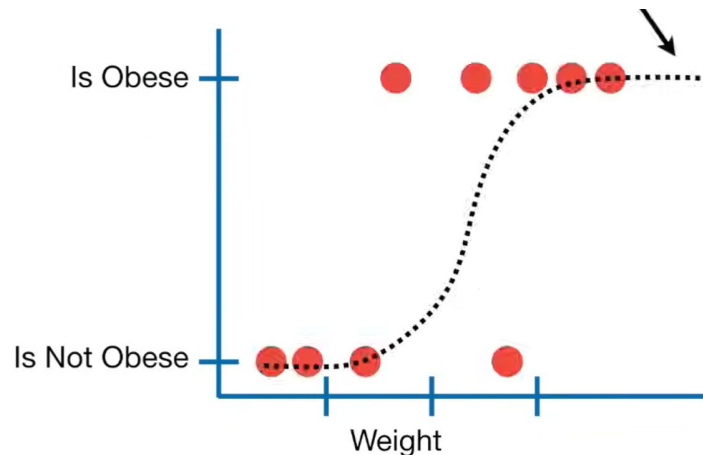
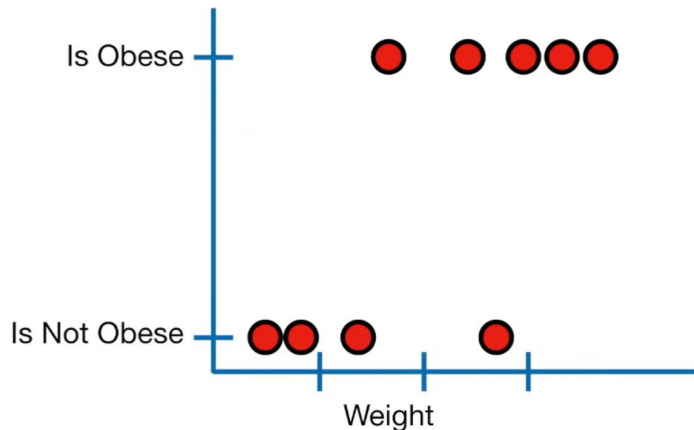
Logistic Regression

Logistic regression is a supervised machine learning algorithm widely used for **binary classification** tasks



Logistic Regression

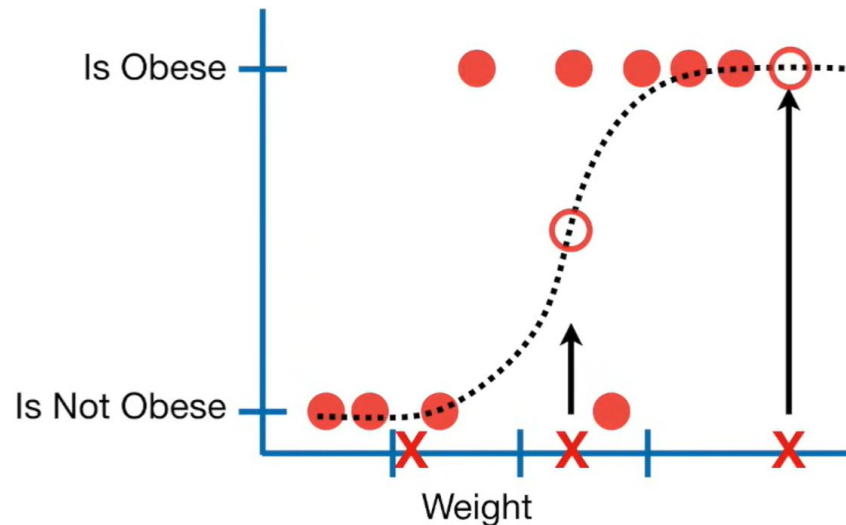
Instead of fitting a line to the data, logistic regression fits an “S” shaped logistic (sigmoid) function. The logistic function is transform a linear combination of input features into a **probability value** ranging between 0 and 1.



Logistic Regression

The curve represents the **probability** of a specific outcome occurring based on the given input variable.

For example, if the probability a mouse is obese is $> 50\%$, then we'll classify it as obese, otherwise we'll classify it as "not obese".



Logistic Regression

Mathematical Foundation

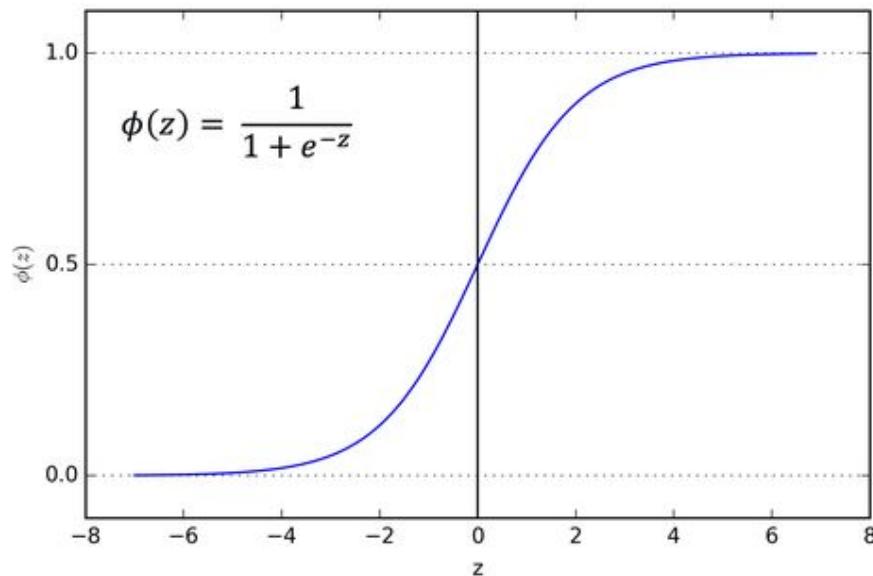
1. Logistic regression starts by calculating a linear combination of the input features: $y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots \dots \dots \beta_n X_n$
2. Logistic (Sigmoid) Function: The linear output (y) is passed through a sigmoid function to transform it into a probability:

Predicted probability of the positive class. $\sigma(z) = \frac{1}{1 + e^{-z}}$

3. Threshold Decision:
If $\sigma(z) > 0.5$, predict class 1.
Otherwise, predict class 0.

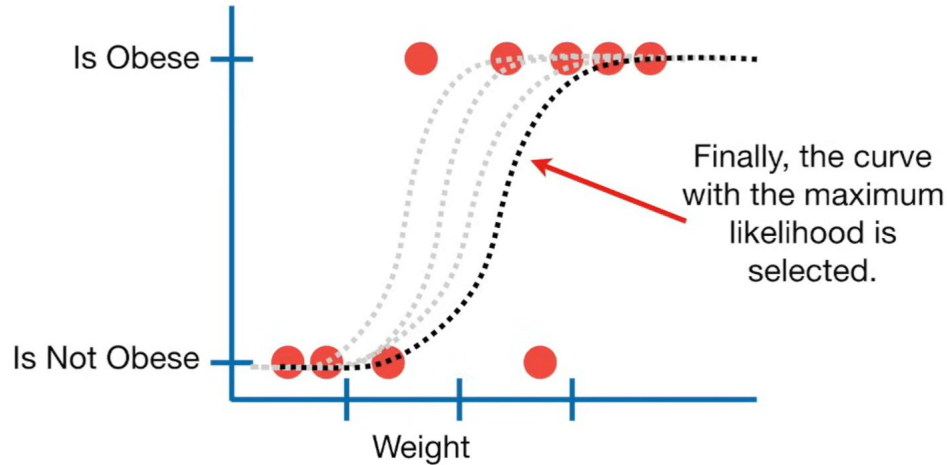
Logistic Regression

The sigmoid function transforms the continuous real number into a range of (0,1)



Logistic Regression

One major difference between linear regression and logistic regression is how the line is fit to the data. In logistic regression, there is no residual, so you cannot use R-squared and other familiar metrics. Instead, maximum likelihood estimation is used.



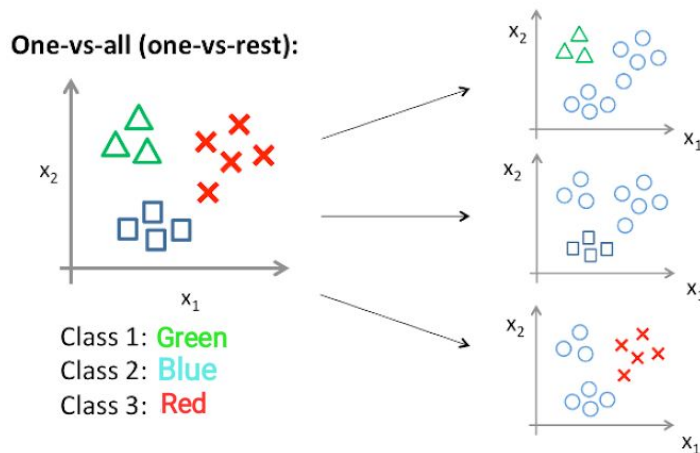
Log likelihood function:
$$L(\beta) = \sum_{i=1}^n [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Multiclass Logistic Regression

Logistic regression is often associated with **binary classification**, but it can also handle **multiclass classification** using techniques like **One-vs-Rest (OvR)** or Softmax Regression.

One-vs-Rest (OvR):

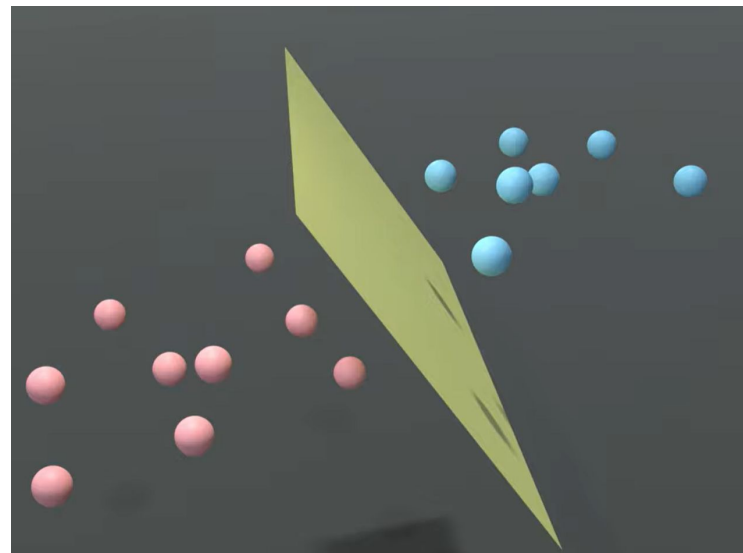
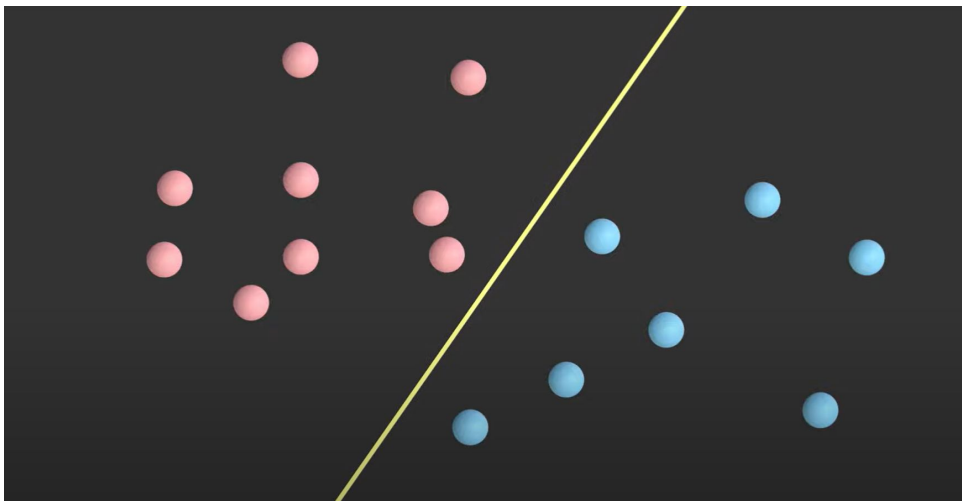
- Logistic regression is applied separately for each class.
- The model predicts the probability of each class versus the rest.
- The class with the highest probability is chosen as the prediction.



Support Vector Machines

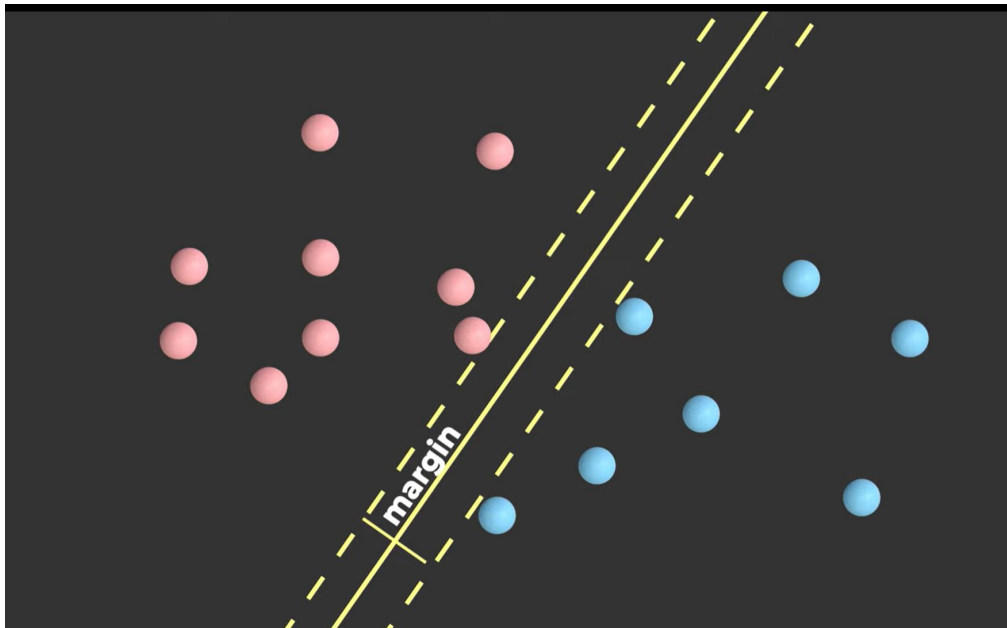
(svm)

Linear Support Vector Machines



Linear Support Vector Machines

Margin is the perpendicular distance between the closest data points and the Hyperplane (both sides). The closest points where the margin distance is calculated are considered as the **support vectors**.



SVMs choose the linear separator with the largest margin

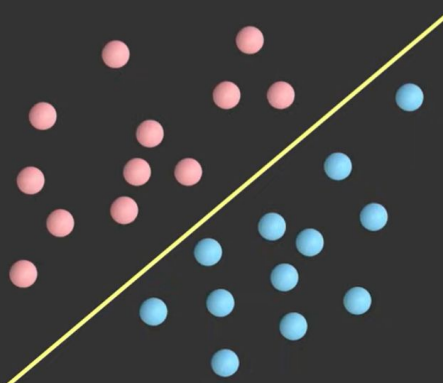
Linear Support Vector Machines

Convex Optimization Problem

$$\max_{w, b} ||w||^{-2}$$

$$w^T x + b \geq 1 \quad \forall x \in C_1$$

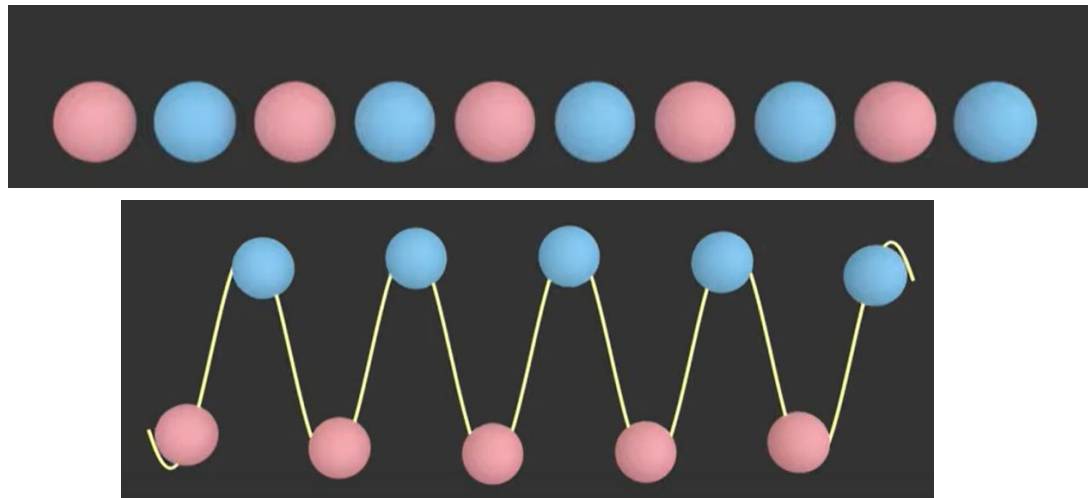
$$w^T x + b \leq -1 \quad \forall x \in C_2$$



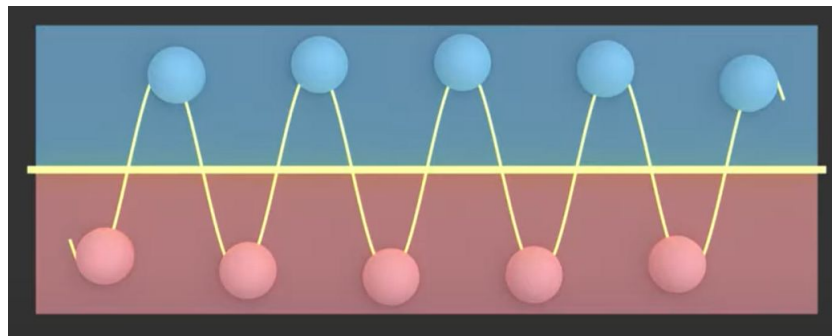
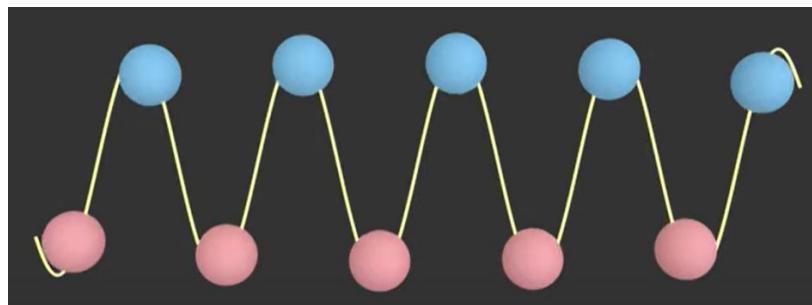
Support Vector Machines



Support Vector Machines

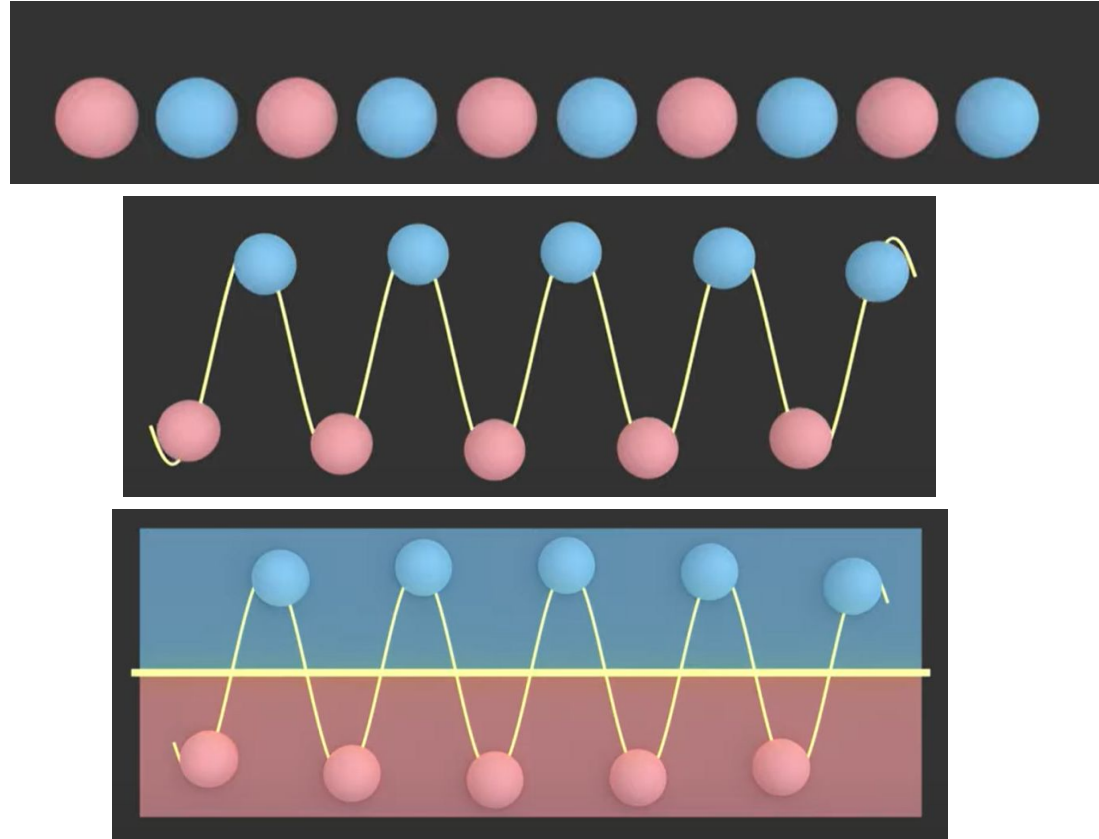


Support Vector Machines



Support Vector Machines

In SVM, a **kernel function** is used to transform data into a higher-dimensional space where it might become linearly separable. Kernels allow SVM to create **non-linear decision boundaries** by mapping the original features into a higher dimension without explicitly computing the coordinates in that space (known as the "kernel trick").



Support Vector Machines

```
from sklearn import svm
```

```
# data points
```

```
X = [  
[ 0.1, 2.3],  
[-1.5, 2.5],  
[ 2.0, -4.3],  
#...  
]
```

```
# labels
```

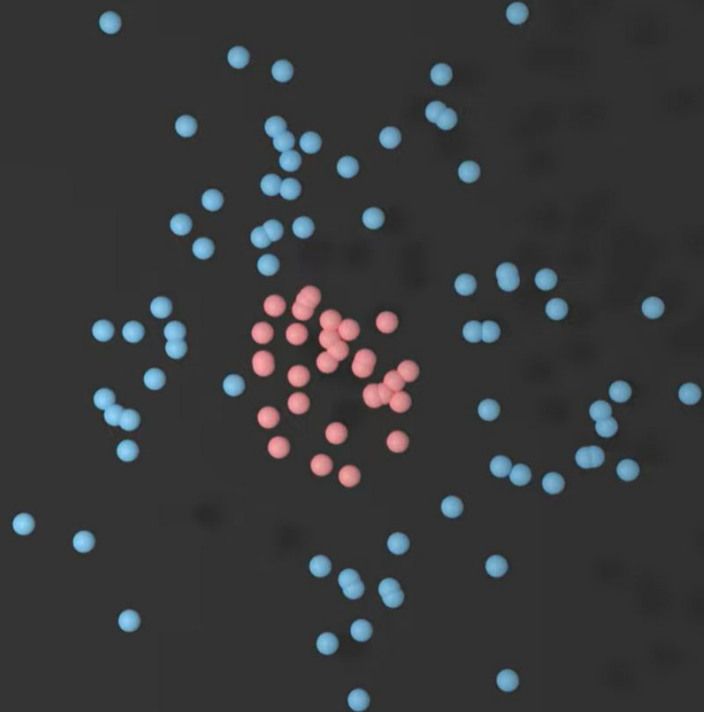
```
y = [0, 1, 0, ] # ...
```

```
# nonlinear transformation
```

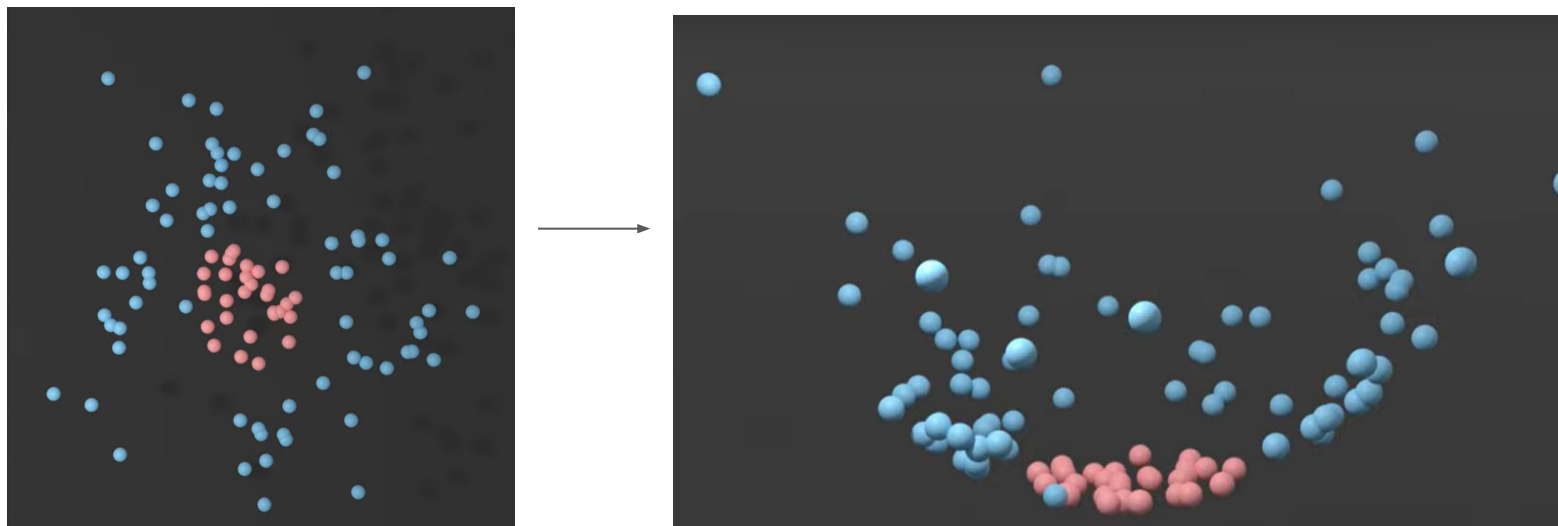
```
fX = [ (x[0], x[1], x[0]**2+x[1]**2)  
      for x in X]
```

```
# fit
```

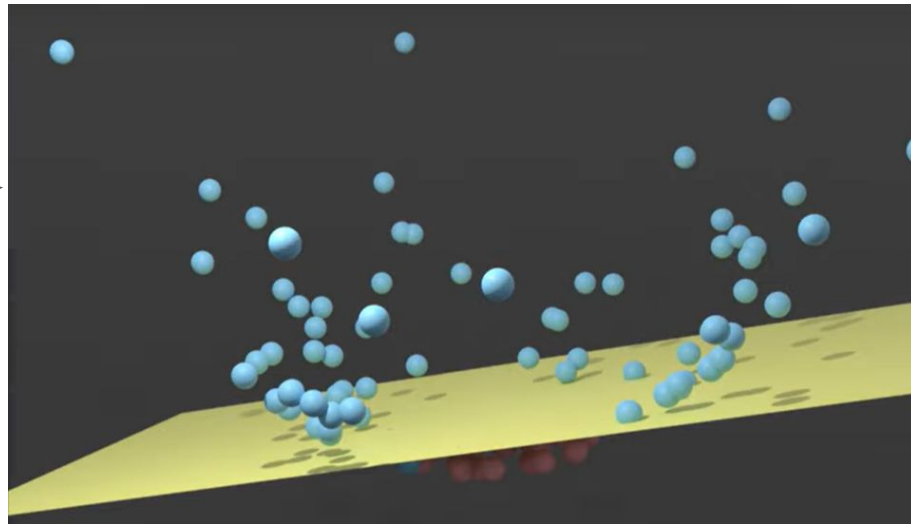
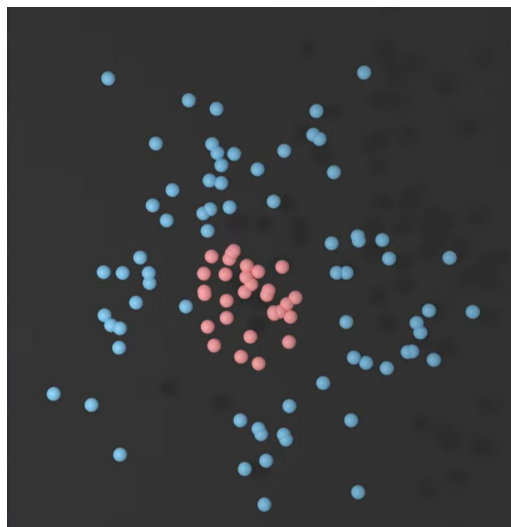
```
svm.SVC().fit(fX, y)
```



Support Vector Machines



Support Vector Machines



The Kernel Trick

The kernel trick **replaces the explicit computation** of the dot product between transformed data points in the higher-dimensional space with a kernel function that **calculates this dot product directly**.

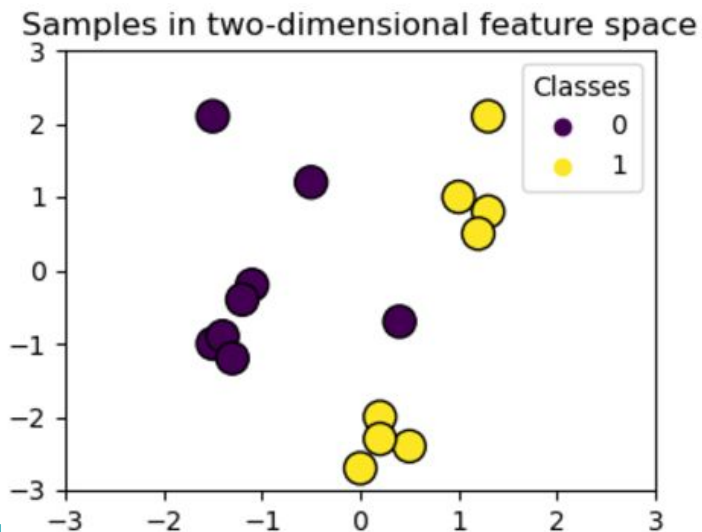
For example:

Instead of computing $\phi(x_i) \cdot \phi(x_j)$, where $\phi(x)$ maps x to the higher-dimensional space, the kernel function $K(x_i, x_j)$ is used:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

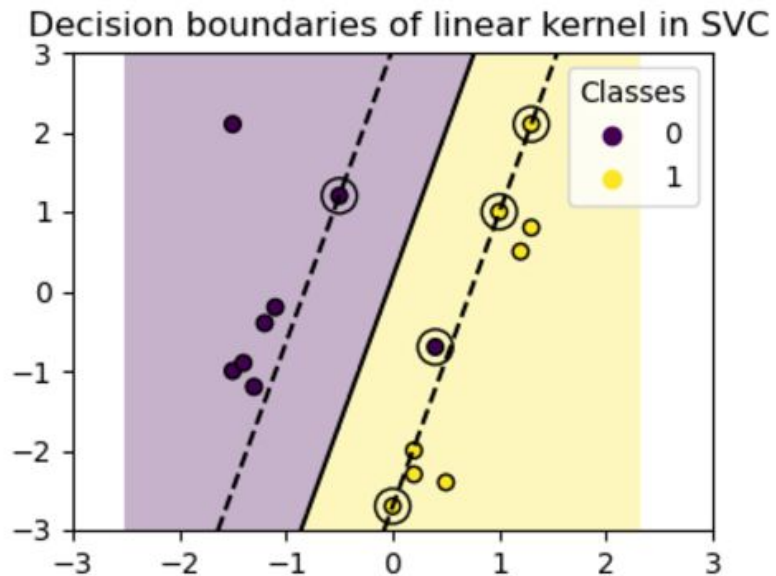
$K(x_i, x_j)$ is computed in the original feature space, avoiding the explicit mapping to the higher-dimensional space.

Common Kernels



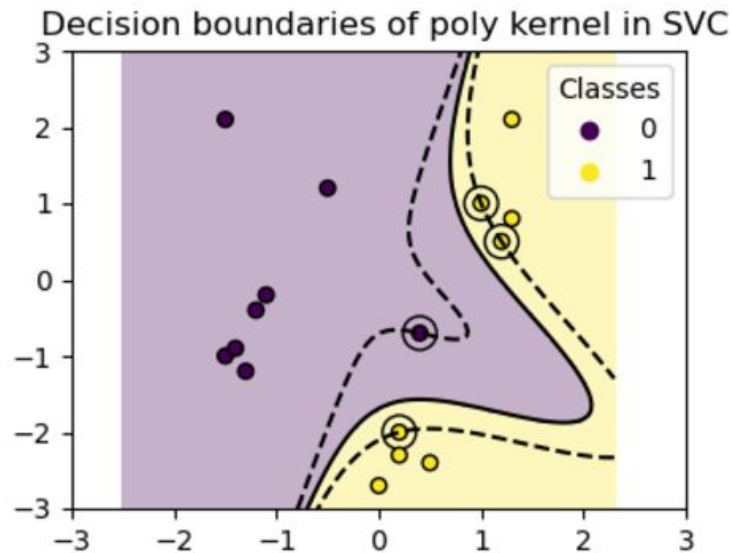
Common Kernels

- Linear Kernel: $K(x_i, x_j) = x_i \cdot x_j$



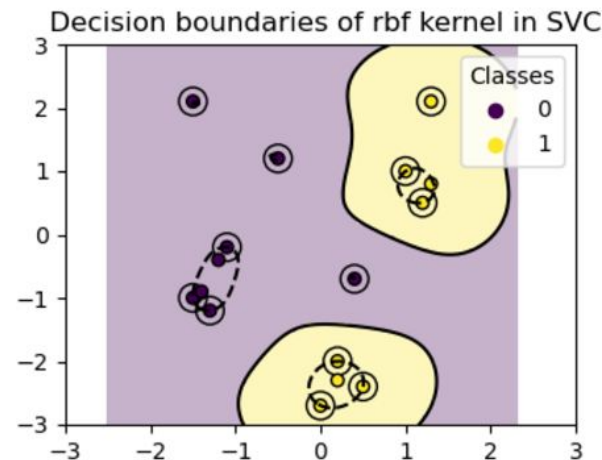
Common Kernels

- Linear Kernel: $K(x_i, x_j) = x_i \cdot x_j$
- Polynomial Kernel: $K(x_i, x_j) = (x_i \cdot x_j + c)^d$



Common Kernels

- Linear Kernel: $K(x_i, x_j) = x_i \cdot x_j$
- Polynomial Kernel: $K(x_i, x_j) = (x_i \cdot x_j + c)^d$
- Radial Basis Function (RBF) Kernel: $K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$



The Kernel Trick

For students interested in a deeper understanding of **Support Vector Machines (SVM)** and the **kernel trick**, here are some additional videos and articles that offer both intuitive explanations and mathematical depth:

<https://www.youtube.com/watch?v=efR1C6CvbmE&t=691s>

<https://www.youtube.com/watch?v=Toet3EiSFcM>

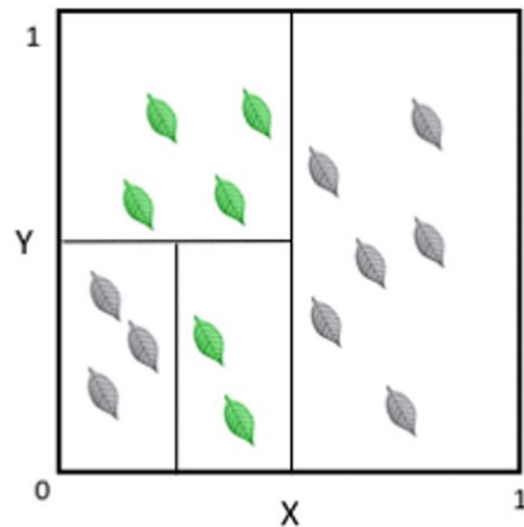
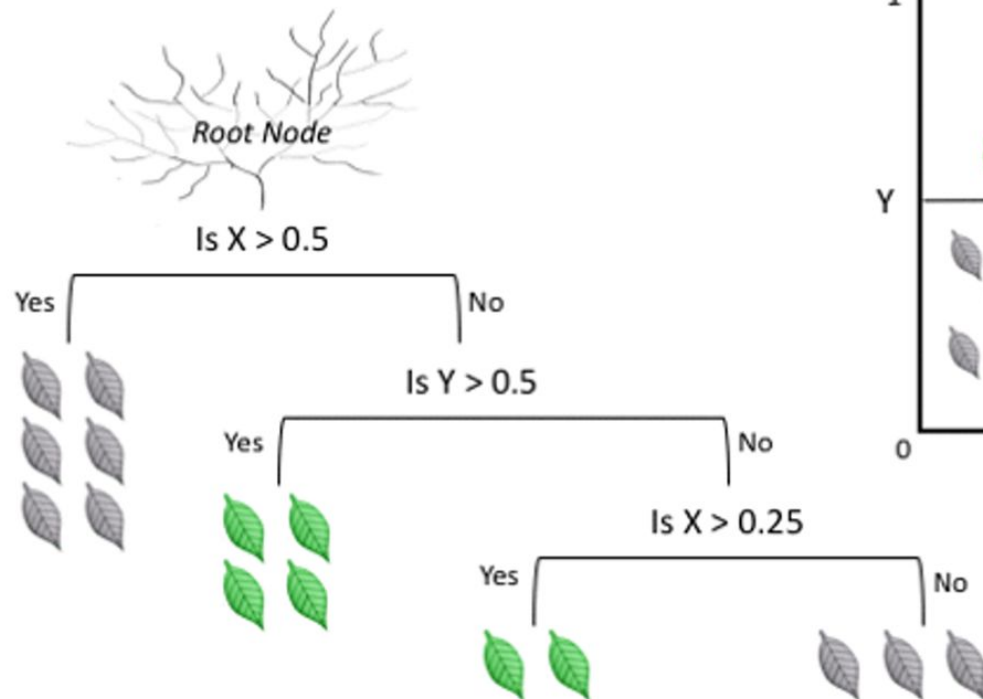
https://www.youtube.com/watch?v=Qc5IyLW_hns&t=612s

<https://www.youtube.com/watch?v=16nF6g7C4q8>

<https://medium.com/@abgajawa/understanding-the-kernel-trick-eb047958cab3>

Decision Tree

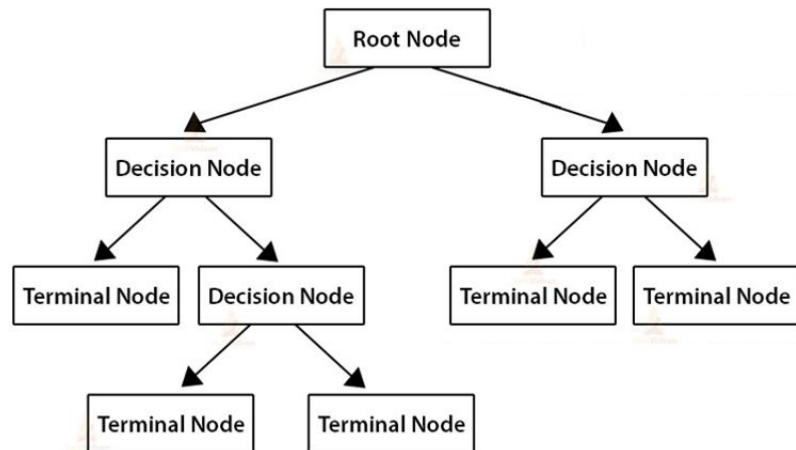
Decision Tree



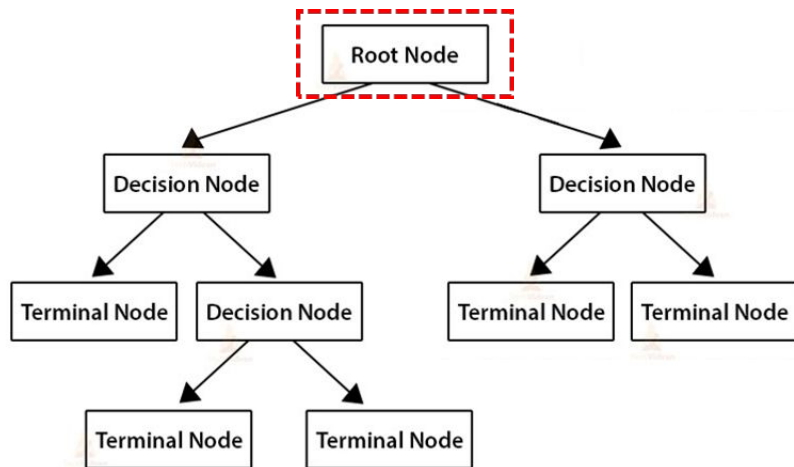
Decision Tree

- Is one of the supervised machine learning algorithms.
- This algorithm can be used for both regression and classification problems.
- A decision tree follows a set of if-else conditions to visualize the data and classify it based on these conditions.

Decision Tree



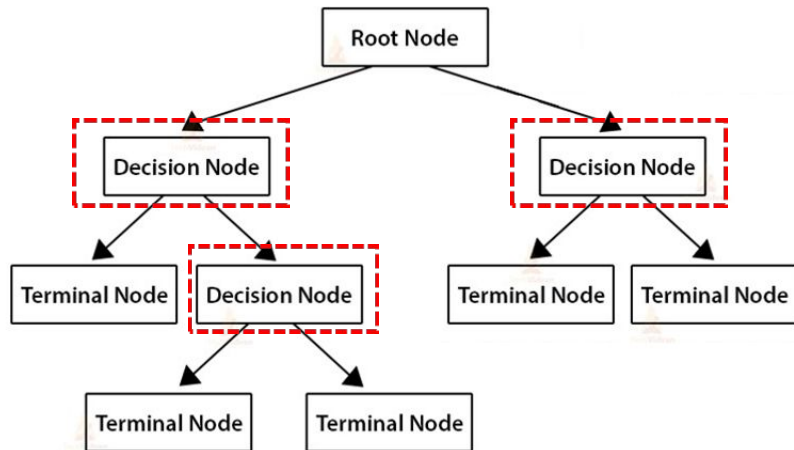
Decision Tree



Root Node:

- The top-most node of a decision tree.
- It does not have any parent node.
- It represents the entire population or sample
- Dividing according to various features.

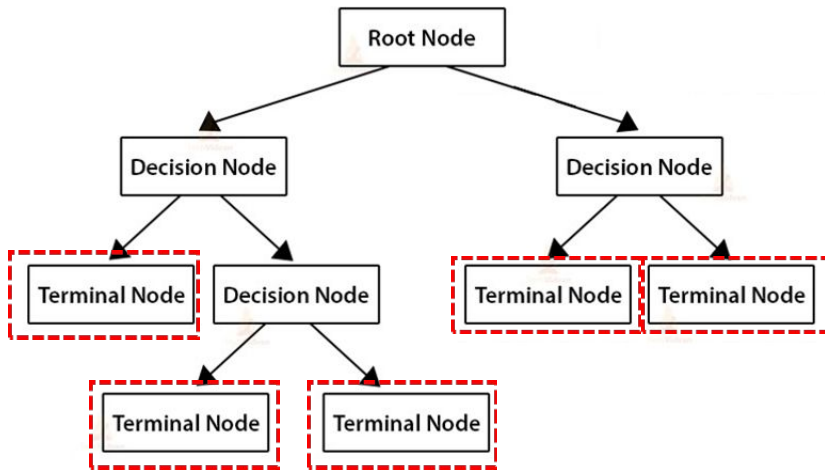
Decision Tree



Decision Node:

- When a sub-node splits into further sub-nodes, then it is called a decision node.

Decision Tree



Leaf/Terminal Node:

- This is the end of the decision tree where it cannot be split into further sub-nodes.
- Leaf node represents a classification or decision.

Decision Tree

Entropy (Information Gain)

To decide the best split at each node, decision trees use metrics that measure the "purity" of the resulting groups. The most common metric:

Decision Tree

Entropy

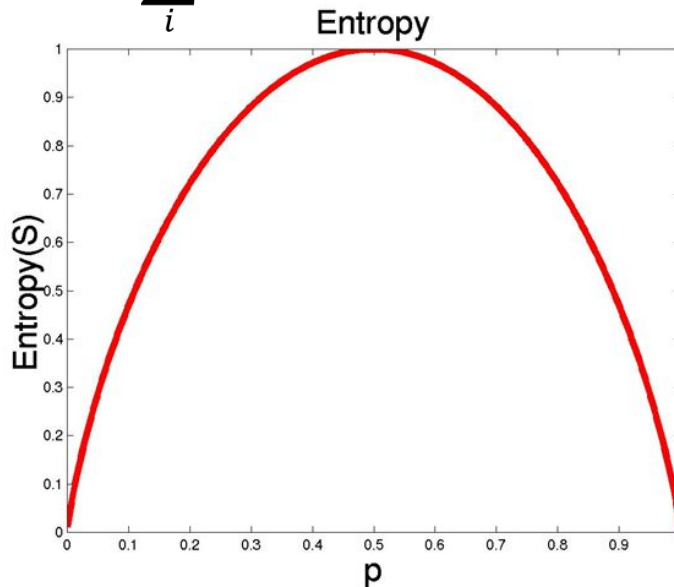
- Entropy is a **measure of uncertainty or randomness in a dataset**.
In the context of decision trees, entropy is used to quantify how mixed or pure a dataset is with respect to its target classes.
- A **lower** entropy indicates a more "**pure**" set where one class is dominant, while a **higher** entropy suggests a more **balanced** mix of classes.

Decision Tree

Entropy

The entropy S of a dataset with a probability distribution $p_1, p_2, \dots, p_n$ is given by:

$$Entropy(S) = - \sum_i p_i \log_2 p_i = -p_+ \log_2 p_+ - p_- \log_2 p_-$$



Here, p_i represents the proportion of samples in class i

Decision Tree

Entropy

$$Entropy(S) = - \sum_i p_i \log_2 p_i = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

- Suppose we throw a fair coin ($p_{Tail} = 0.5, p_{Head} = 0.5$)
 - $Entropy(S) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$ **High uncertainty**
- Suppose we throw a unfair coin ($p_{Tail} = 0.99, p_{Head} = 0.01$)
 - $Entropy(S) = -0.99 \log_2 0.99 - 0.01 \log_2 0.01 = 0.08$ **Low uncertainty**

Decision Tree

Information Gain

Definition: Information Gain (IG) measures the reduction in entropy (uncertainty) achieved by splitting a dataset based on a specific feature.

Purpose: The goal is to split the data in a way that **increases "purity" of each subset**, making it easier to classify examples.

Information Gain helps determine which feature to split on at each step of the tree-building process.

Decision Tree

Information Gain

How Information Gain Works in Decision Trees:

1. Calculate Entropy of the dataset before the split
2. Split the Dataset
3. Calculate Entropy After the Split
4. Compute Information Gain
5. Choose the Feature with Highest Information Gain

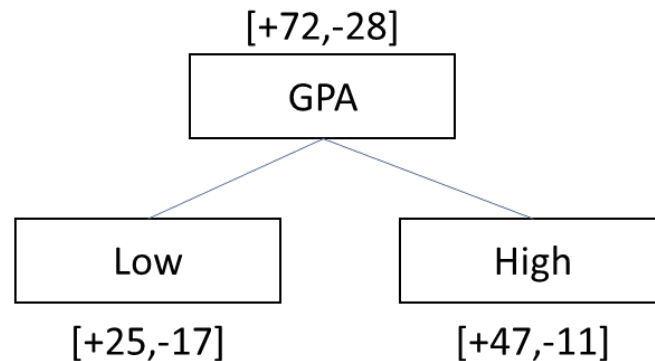
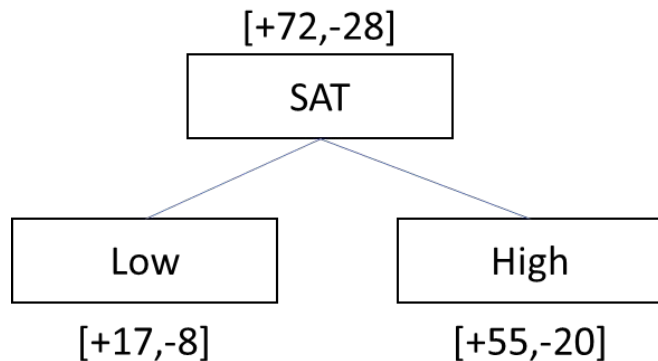
$$IG(S, A) = \underbrace{Entropy(S)}_{\text{The entropy of S (before splitting)}} - \underbrace{\sum_{v \in Values(A)} \frac{|S_v|}{|S|} \cdot Entropy(S_v)}_{\text{The entropy sum of S after splitting according to feature A (After the split)}}$$

Decision Tree

Information Gain

We want to find which feature need to split

Index	GPA	SAT	Graduate BSc Degree
1	High	High	Yes
2	High	Low	No
3	High	High	Yes
.....			

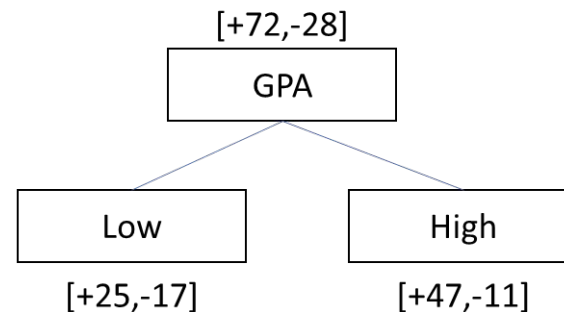
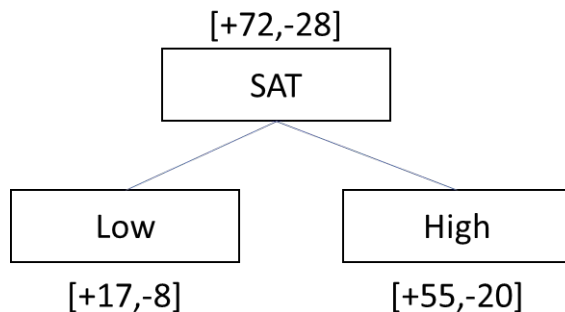


Decision Tree

Information Gain

We want to find which feature need to split

Before the split: $Entropy(S) = Entropy([+72, -28]) = -\frac{72}{100} \cdot \log_2\left(\frac{72}{100}\right) - \frac{28}{100} \cdot \log_2\left(\frac{28}{100}\right) = 0.855$



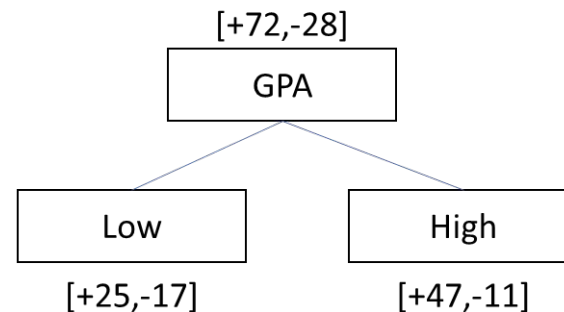
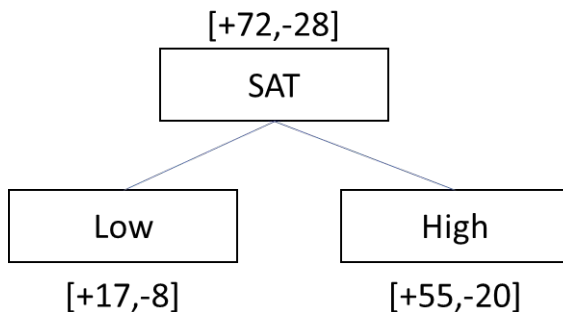
Decision Tree

Information Gain

We want to find which feature need to split

Before the split: $Entropy(S) = Entropy([+72, -28]) = -\frac{72}{100} \cdot \log_2\left(\frac{72}{100}\right) - \frac{28}{100} \cdot \log_2\left(\frac{28}{100}\right) = 0.855$

$$IG(S, GPA) = 0.855 - \left(\frac{42}{100} (Entropy([+25, -17])) + \frac{58}{100} (Entropy([+47, -11])) \right) = \mathbf{0.04}$$



Decision Tree

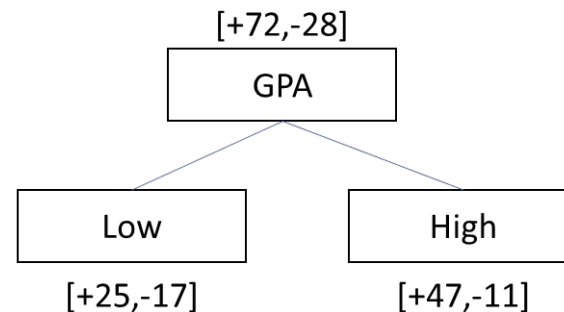
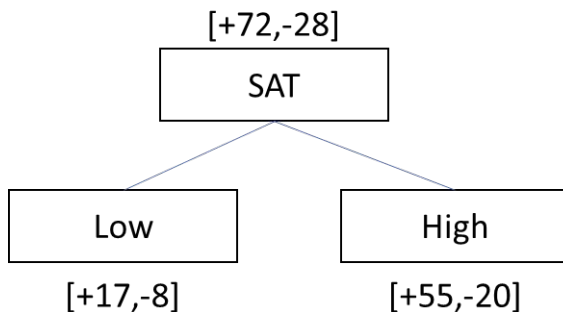
Information Gain

We want to find which feature need to split

Before the split: $Entropy(S) = Entropy([+72, -28]) = -\frac{72}{100} \cdot \log_2\left(\frac{72}{100}\right) - \frac{28}{100} \cdot \log_2\left(\frac{28}{100}\right) = 0.855$

$$IG(S, GPA) = 0.855 - \left(\frac{42}{100} (Entropy([+25, -17])) + \frac{58}{100} (Entropy([+47, -11])) \right) = \mathbf{0.04}$$

$$IG(S, SAT) = 0.855 - \left(\frac{25}{100} (Entropy([+17, -8])) + \frac{75}{100} (Entropy([+55, -20])) \right) = \mathbf{0.0014}$$



Decision Tree

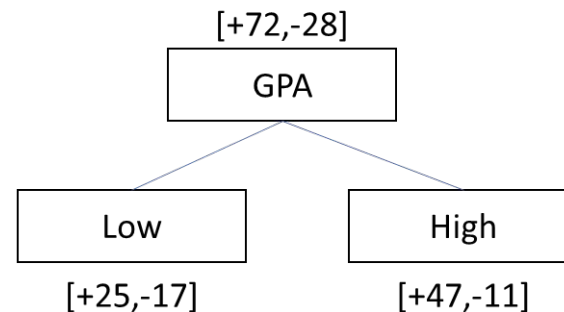
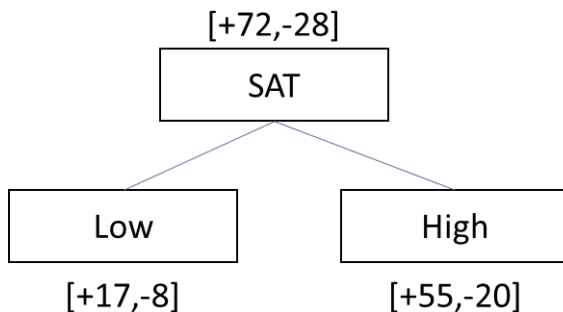
Information Gain

We want to find which feature need to split

Before the split: $Entropy(S) = Entropy([+72, -28]) = -\frac{72}{100} \cdot \log_2\left(\frac{72}{100}\right) - \frac{28}{100} \cdot \log_2\left(\frac{28}{100}\right) = 0.855$

$$IG(S, GPA) = 0.855 - \left(\frac{42}{100} (Entropy([+25, -17])) + \frac{58}{100} (Entropy([+47, -11])) \right) = \mathbf{0.04}$$

$$IG(S, SAT) = 0.855 - \left(\frac{25}{100} (Entropy([+17, -8])) + \frac{75}{100} (Entropy([+55, -20])) \right) = \mathbf{0.0014}$$

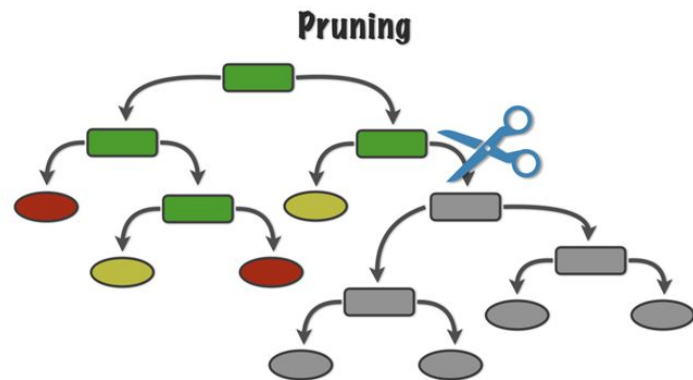


Decision Tree

Pruning

Pruning is the process of reducing the size of a decision tree by converting some branch (decision) nodes into leaf nodes and removing the leaf nodes under those branches.

Pruning is useful because classification trees may fit the training data well but may perform poorly on new data. A simpler tree often helps to avoid overfitting.



Decision Tree

Advantages & Disadvantages

Advantages:

- Decision trees are easy to visualize and easy to explain to technical teams as well as stakeholders.
- Non-linear patterns in the data can be captured easily.
- It requires fewer data preprocessing from the user, for example, there is no need to normalize columns.

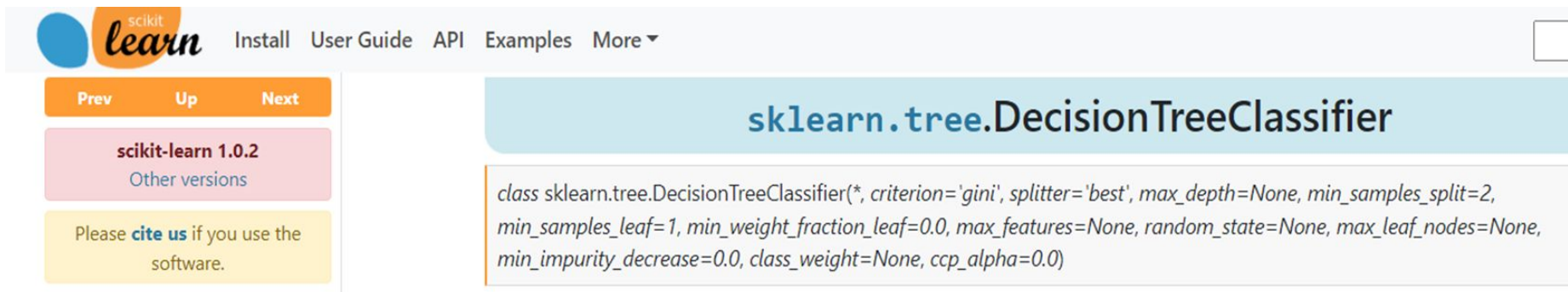
Disadvantages:

- Overfitting of the data is possible.
- The small variation in the input data can result in a different decision tree. (This can be reduced by bagging and boosting algorithms.)
- Decision trees are biased with imbalance dataset, so it is recommended that balance out the dataset before creating the decision tree.

Decision Tree

Code

Documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>



The screenshot shows the scikit-learn website interface. At the top, there's a navigation bar with links: Install, User Guide, API, Examples, and More. Below this, there's a sidebar with navigation buttons: Prev, Up, and Next. The main content area displays the title "sklearn.tree.DecisionTreeClassifier" in a light blue header. Below the title, the class signature is shown: `class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0)`. On the left sidebar, there's a section for "scikit-learn 1.0.2" with a link to "Other versions" and a note to "Please cite us if you use the software."

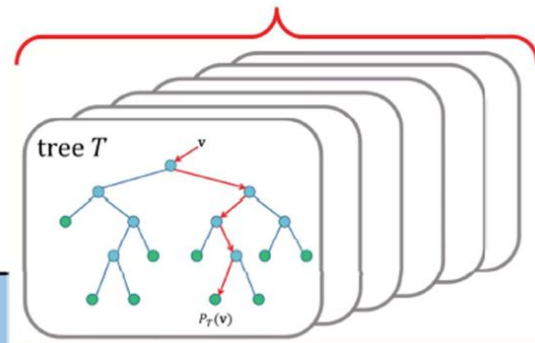
Python Code:

```
from sklearn.datasets import load_iris
from sklearn import tree
iris = load_iris()
X, y = iris.data, iris.target
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, y)
#tree.plot_tree(clf) #uncomment to plot tree
clf.predict(X) #2D data you want to predict
```

Random Forest Classifier - Ensemble Learning

Multiple decision trees built in parallel from random subsets of the data

1. Randomly sample K observations (and features)
2. Build a Decision Tree associated to these K data points
3. Choose the number N_{tree} out of trees you want to build and repeat 1&2
4. For a new data point, make each of the trees predict the category to which the data point belongs, and assign the the new data point to the category that wins the majority vote.



Ensemble Learning

Ensemble learning is a machine learning paradigm that combines multiple models, known as "learners," to improve overall predictive performance.

The fundamental idea is that **by aggregating diverse models**, the ensemble can capture various patterns in the data, leading to more accurate and robust predictions than any single model could achieve.

Advantages: Improved Accuracy, Robustness, Versatility

Challenges: Increased Complexity, Computational Resources, Diminishing Returns

Classification - evaluation

Classification evaluation

Terminology:

		Predicted	
		0	1
Actual	0	TN	FP
	1	FN	TP

True Positives (TP): Correctly identified positive instances

False Positives (FP): Incorrectly identified positive instances

True Negatives (TN): Correctly identified negative instances

False Negatives (FN): Incorrectly identified negative instances

Classification evaluation

Accuracy: measures how often the model correctly predicts the outcome.

$$\text{Accuracy} \Rightarrow \frac{TP+TN}{TP+TN+FN+FP}$$

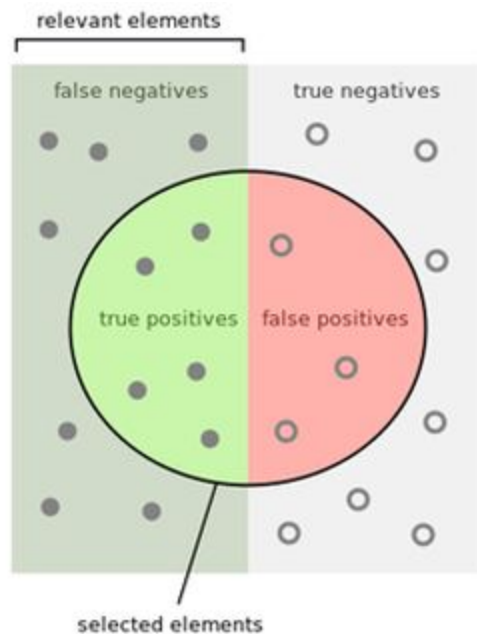
Precision: measures how often a machine learning model correctly predicts the positive class.

$$\text{Precision} \Rightarrow \frac{TP}{TP+FP}$$

Recall: measures how often a machine learning model correctly identifies positive instances (true positives) from all the actual positive samples in the dataset.

$$\text{Recall} \Rightarrow \frac{TP}{TP+FN}$$

Classification evaluation



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Classification evaluation

Case Study:

Accuracy: In scenarios where the classes are relatively balanced, and the misclassification cost is the same for each class, accuracy can be a reliable metric.

Weather Prediction

Precision: Precision is important when false positives are costly.

Financial fraud detection

Recall: Recall is important in scenarios where False Negatives are costly.

COVID-19 tests

Classification evaluation

F1 Score:

F1 score **balances** between precision and recall.

It does this because it's computed as the **harmonic mean** of precision and recall

$$\text{F1 Score} = \frac{2 \cdot (\text{precision} \times \text{recall})}{\text{precision} + \text{recall}}$$

$$\text{F1 Score} = \frac{2TP}{2TP + FP + FN}$$

Classification evaluation

F1 Score:

F1 score **balances** between precision and recall.

It does this because it's computed as the **harmonic mean** of precision and recall

$$\text{F1 Score} = \frac{2 \cdot (\text{precision} \times \text{recall})}{\text{precision} + \text{recall}}$$

$$\text{F1 Score} = \frac{2TP}{2TP + FP + FN}$$

So many more... Further reading:

https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

הטכניון
בית"ס ללימודי המשך
של הטכניון ע"ש עזריאלי

A husky dog with grey and white fur is standing on a green lawn. The dog is looking upwards and to the left with its mouth slightly open. Its tail is bushy and white, curled upwards. The background is a blurred green field with some trees in the distance.

		0	1	2	3	4	5	6	7	8	9
Dog 1	0: Chihuahua (79)	17	1	2	13	18	6	7	8	4	3
	1: Japanese_spaniel (93)	4	54	5	0	9	7	4	7	3	0
	2: Maltese_dog (131)	3	1	90	5	13	5	2	11	1	0
	3: Pekinese (76)	5	0	9	20	21	4	9	6	1	1
	4: Shih-Tzu (103)	5	4	12	12	33	2	10	20	4	1
	5: Blenheim_spaniel (95)	5	8	1	1	9	52	4	14	0	1
Dog 2	6: Malamute (90)	3	0	9	4	19	2	8	42	1	2
	7: Siberian_husky (96)	4	0	6	5	6	6	21	40	7	1
	8: Boston_bull (77)	4	2	5	3	12	3	3	16	25	4
	9: French_bulldog (86)	2	3	9	7	16	3	9	13	21	3

Hands on !



Open Google colab notebook -
ML5 - Classification